

Software Engineering Lab 4

VibeCoding – Claude Chat History

Field	Details
Lab No.	4
Lab Title	VibeCoding
Course ID	UE24CS341A
Student Name	Nithin Kumar M
SRN	PES1UG25CS829

Complete captured chat history for Task 1, Task 2, Task 3 and Task 4.

I am doing a Software Engineering VibeCoding lab.

I have uploaded the files from my assigned Word Scramble Repair Lab project.

First, read README.md and inspect the uploaded Python files.

We are working on TASK 1 ONLY.

TASK 1:

Fix the deliberate guess-comparison bug in `game/game_engine.py`.

The current `submit_guess()` function incorrectly compares the user's guess with `self.scrambled_word`.

Change the comparison so that the user's guess is compared with `self.secret_word`.

Expected behavior:

1. If the user enters the correct unscrambled word:

- increase the score by 1
- show the success feedback
- move to the next round

2. If the guess is incorrect:

- show the existing wrong-guess feedback
- clear the input
- do not advance the round

3. If the input is empty:

- keep the existing empty-input behavior
- do not apply a penalty

IMPORTANT:

- Do NOT implement Task 2, Task 3, or Task 4 yet.
- Do NOT redesign the UI.
- Do NOT change unrelated code.
- Make the smallest necessary change for Task 1.
- Tell me exactly which file you changed and what you changed.
- Show me the relevant code change before applying it.

Show less

s ago

Yes, apply the Task 1 fix.

Keep the old BUG SYMPTOM comment removed, as you proposed.

Do not make any other changes.

files >



Game engine

PY

Download

Now implement Task 2 from the README.

Add a clickable HINT button beside the SUBMIT button.

Requirements:

- The HINT button must be clearly visible beside SUBMIT.
- When the player clicks HINT, reveal the first unrevealed letter of `self.secret_word` in its correct position.
- The hint should be displayed clearly in the game UI.
- Deduct a small point penalty when a hint is used.
- Make sure the existing Task 1 fix remains unchanged.
- Do not implement Task 3 or Task 4 yet.
- Do not modify unrelated files or features.
- After making the change, explain exactly which files were changed and how the hint works.

Now implement Task 3 from the README only.

Add a countdown timer to the Word Scramble game.

Requirements:

- Add a 20-second countdown timer for each round.
- Display the remaining time clearly in the game UI.
- The timer must reset to 20 seconds whenever `next_round()` starts a new round.
- While the player is solving the word, the timer should count down normally.
- When the timer reaches 0:
 1. Reveal the correct secret word.
 2. Display "TIME'S UP!" clearly.
 3. Automatically transition to the next round by calling `self.next_round()`.
- Make sure the timer does not keep running incorrectly after the round changes.
- Make sure the existing Task 1 fix remains unchanged.
- Make sure the existing Task 2 HINT functionality remains unchanged.
- Do not implement Task 4 yet.
- Do not modify unrelated files or features.
- Keep the implementation simple and appropriate for the existing Pygame code.

After making the change, explain:

1. Which files you changed.
2. How the countdown works.
3. What happens when the timer reaches 0.
4. How you ensured Task 1 and Task 2 still work.

Show less

2 hours ago   



Now implement Task 4 from the README.

Task 4 requirements:

- Render each scrambled letter as an individual graphical square tile instead of plain text.
- Allow the player to click and drag the letter tiles.
- Provide a rearrangement rack where the player can arrange the letters into the desired order.
- The existing Task 1 correct-guess fix must remain.
- The existing Task 2 HINT functionality must remain.
- The existing Task 3 countdown timer must remain.
- Do not remove or break SUBMIT, HINT, scoring, feedback, wrong guesses, empty input handling, or automatic next-round behavior.
- Make the smallest clean changes necessary.
- Tell me exactly which files you changed.
- Do not modify unrelated files.
- After implementing it, explain how I should test the drag-and-drop behavior.

Show less

2 hours ago



I read the README and all the uploaded files (`main.py`, `game_engine.py`, `text_box.py`). I ignored the two `.pyc` files, since they're just compiled copies. Task 1 only, nothing else touched. 🙌

The problem

The bug is in `game_engine.py`, inside `submit_guess()`.

When a round starts, the game keeps two versions of the word:

- `self.secret_word` is the real answer, like `PYTHON`
- `self.scrambled_word` is the jumbled version, like `OHTPNY`

Right now the check is `guess == self.scrambled_word` ... so the game only says "CORRECT" if you type the jumbled letters back exactly as shown 🤖 If you actually unscramble it and type `PYTHON`, it says "WRONG GUESS!". Basically the game is grading you for copying, not for thinking. 😬

The proposed change

File: `game/game_engine.py`, function: `submit_guess()`

Before:

```
python

# BUG SYMPTON:
# Player's guess is validated against the scrambled text instead of the original
is_correct = (guess == self.scrambled_word)
```

After:

```
python

is_correct = (guess == self.secret_word)
```

The comparison changes from `self.scrambled_word` to `self.secret_word`. I also removed the two-line `BUG SYMPTON` comment, because it describes a bug that will no longer exist. If you'd rather keep your diff to a single line, I can leave the comment in place. Your call.

Why nothing else needs to change

The rest of the function already does what you asked:

- Empty input: hits the early `return` with "Type a word before submitting!" and no score change, so no penalty. ✅
- Correct guess: `score += 1`, shows the success message, calls `next_round()`. ✅
- Wrong guess: shows "WRONG GUESS! Try again.", clears the box, and does not call `next_round()`. ✅

So the fix really is that one comparison.



Yes, apply the Task 1 fix.

Keep the old BUG SYMPTOM comment removed, as you proposed.

Do not make any other changes.

Ran a command, shared files >

Applied. ✅ Task 1 is done, nothing else touched.

File changed: `game/game_engine.py`, in `submit_guess()`

Before:

```
python

# BUG SYMPTOM:
# Player's guess is validated against the scrambled text instead of the original
is_correct = (guess == self.scrambled_word)
```

After:

```
python

is_correct = (guess == self.secret_word)
```

I ran a diff against your original, and that's the only difference: the two comment lines are gone and the comparison now uses `self.secret_word`. Line endings and the rest of the file are exactly as they were. 🐛

Put this file back at `game/game_engine.py` in your project folder, replacing the old one. Then run `python main.py`, type the unscrambled word, and it should say CORRECT and move on.



For your "after" video, show three things: a correct guess (score goes up and a new word appears), a wrong guess (warning, box clears, same word stays), and an empty submit (prompt only, no penalty).

Ready for Task 2 whenever you are. 😊



Game engine
PY

Download

Now implement Task 2 from the README.

Add a clickable HINT button beside the SUBMIT button.

Requirements:

- The HINT button must be clearly visible beside SUBMIT.
- When the player clicks HINT, reveal the first unrevealed letter of `self.secret_word` in its correct position.
- The hint should be displayed clearly in the game UI.
- Deduct a small point penalty when a hint is used.
- Make sure the existing Task 1 fix remains unchanged.
- Do not implement Task 3 or Task 4 yet.
- Do not modify unrelated files or features.
- After making the change, explain exactly which files were changed and how the hint works.

Ran 2 commands, shared files >

Task 2 is done, and it's the same single file, so replace `game/game_engine.py` in your project with this one. 💡 I tested it headlessly, with no window opened, and it behaved correctly. I haven't seen it on a real screen.

File changed: `game/game_engine.py` only. `main.py` and `text_box.py` are untouched. The Task 1 fix (`guess == self.secret_word`) is still there exactly as before.

What I added

- `__init__` : a `HINT` button rectangle right next to `SUBMIT` (an orange one, so it stands out from the green). It also sets `hint_penalty = 1` and a counter `hints_used = 0`.
- `next_round()` : resets `hints_used` to 0, so every new word starts with no hints.
- `use_hint()` : a new method that runs on each click.
- `handle_event()` : a new `elif` so clicking `HINT` calls `use_hint()`.
- `render()` : draws the `HINT` button and, once you've used a hint, draws the hint pattern below the feedback message.

How the hint works

The game only remembers one number, `hints_used`. Say the word is PYTHON:

- Click `HINT` once and it shows `P _ _ _ _ _`.
- Click again and it shows `P Y _ _ _ _`.
- Each click reveals the next letter in its correct position, always left to right. 📺

Each click costs 1 point, and the feedback line says "Hint used! -1 point" in amber. The score never drops below 0, so you can't go negative. Once the whole word is revealed, more clicks just say so and don't cost anything.

I tested these cases, and all passed:

- Hints reveal in order and cost 1 point each.
- A correct guess still scores and moves on, and hints reset for the new word.
- A wrong guess still clears the box and keeps the same word.

- A correct guess still scores and moves on, and hints reset for the new word.
- A wrong guess still clears the box and keeps the same word.
- The score stays at 0 instead of going negative.
- The buttons don't overlap, and HINT ends at x=595, inside the 700px window.

Design choices you can change

The penalty is 1 point, set by `self.hint_penalty` in `__init__`. If the README's "small penalty" means something different to your instructor, change that one number. Also, the hint pattern only appears after the first click, so the screen stays clean until someone needs help.

For your video, click HINT twice, then type the word and submit. 🧠 Ready for Task 3 when you are. 😊



Game engine

PY

Download

New! Implement Task 3 from the README only

Now Implement Task 3 from the README only.

Add a countdown timer to the Word Scramble game.

Requirements:

- Add a 20-second countdown timer for each round.
- Display the remaining time clearly in the game UI.
- The timer must reset to 20 seconds whenever `next_round()` starts a new round.
- While the player is solving the word, the timer should count down normally.
- When the timer reaches 0:
 1. Reveal the correct secret word.
 2. Display "TIME'S UP!" clearly.
 3. Automatically transition to the next round by calling `self.next_round()`.
- Make sure the timer does not keep running incorrectly after the round changes.
- Make sure the existing Task 1 fix remains unchanged.
- Make sure the existing Task 2 HINT functionality remains unchanged.
- Do not implement Task 4 yet.
- Do not modify unrelated files or features.
- Keep the implementation simple and appropriate for the existing Pygame code.

After making the change, explain:

1. Which files you changed.
2. How the countdown works.
3. What happens when the timer reaches 0.
4. How you ensured Task 1 and Task 2 still work.

Show less

2 hours ago   

Ran 3 commands, shared files >

Task 3 is done, and I tested it headlessly (no window opened). 🤖 I haven't watched it on a real screen, so check the layout when you run it.

1. Files changed

Only `game/game_engine.py`. I diffed it against the Task 2 version, and everything I did is an addition, apart from one line: `update()` was `pass` and now holds the timer logic.

2. How the countdown works

- **Start time:** `next_round()` saves the current clock time in `round_start_ms` and sets the timer to 20 seconds.
- **Each frame:** `update()` already runs every frame from `main.py`. It checks how much real time has passed since the round began and sets `time_left = 20 - elapsed`.
- **Why real time:** I used `pygame.time.get_ticks()` (the real clock) instead of subtracting a little each frame. That way the timer stays accurate even if the game lags. 🤖
- **On screen:** a "Time left: 15s" label and a bar sit at the bottom. The bar drains as time runs out and changes color: green above half, amber above a quarter, red below that.

out and changes color green above half, amber above a quarter, red below that.

3. What happens at 0

`update()` calls the new `time_up()` method, which does three things:

1. It sets the message to `TIME'S UP! The word was 'PLANET'.` in red, which reveals the answer.
2. It calls `self.next_round()`.
3. That gives you a new word, a fresh 20 seconds, cleared hints and an empty input box.

The message stays on screen through the next round until you do something, so you have time to read the answer. The score is not changed on a timeout.

Why it doesn't keep running wrongly: `next_round()` restarts the clock immediately, so the next frame sees about 20 seconds left and `time_up()` can't fire twice. The timer also resets after a correct guess (it goes through `next_round()`). A wrong guess does not reset it, because the round hasn't changed.

4. How Task 1 and Task 2 still work

- **Task 1:** the line `is_correct = (guess == self.secret_word)` is untouched. I tested that a correct guess scores, moves on and resets the timer, and that a wrong guess clears the box, keeps the word and leaves the timer running.
- **Task 2:** the HINT button, `use_hint()` and the hint display are untouched. I tested a hint click mid-countdown and got the letter reveal and the -1 point. A timeout also resets `hints_used` through `next_round()`, so hints don't carry over to the new word.

Tip for your video: let one word run out to show TIME'S UP. 🕒 If 20 seconds feels too long for a 10-second clip, change `self.round_seconds = 20` in `__init__`, and the bar and label adjust on their own.

Ready for Task 4 whenever you are. 🤖



Game engine
PY

Download

📄 🔄 🌟 🗨️ ⌂ 2 hours ago

Now implement Task 4 from the README.

Task 4 requirements:

- Render each scrambled letter as an individual graphical square tile instead of plain text.
- Allow the player to click and drag the letter tiles.
- Provide a rearrangement rack where the player can arrange the letters into the desired order.
- The existing Task 1 correct-guess fix must remain.
- The existing Task 2 HINT functionality must remain.
- The existing Task 3 countdown timer must remain.
- Do not remove or break SUBMIT, HINT, scoring, feedback, wrong guesses, empty input handling, or automatic next-round behavior.
- Make the smallest clean changes necessary.
- Tell me exactly which files you changed.
- Do not modify unrelated files.
- After implementing it, explain how I should test the drag-and-drop behavior.

Show less

2 hours ago   

Ran 3 commands, read a file, shared files >

Task 4 is done. 🌟 I tested it headlessly with simulated mouse events and checked a rendered screenshot. That's not the same as dragging with a real mouse, so give it a proper try.

Files changed

Only `game/game_engine.py`, `main.py` and `text_box.py` are untouched. Tasks 1, 2 and 3 code is unchanged, apart from two extra lines in `next_round()` and the wrong-guess branch (explained below).

What you'll see

- The plain scrambled text is now a top row of square tiles, one per letter.
- Below it is a rack: a row of empty slots, one per letter of the word.
- You drag tiles from the top row into the rack, and you can drag them around inside the rack to reorder.
- Dropping a tile on an occupied slot swaps the two tiles.
- Dropping a tile on empty space sends it back where it came from.

How it connects to the game

Every time you move a tile, the rack's letters (left to right) are copied into the input box. So SUBMIT, Enter, wrong guesses and empty submits all work exactly as before, because the game still checks the input box. ✅ Typing into the box still works too.

- **Wrong guess:** the input box clears, as it always did, and the tiles now go back to the top row so the two stay in sync.
- **Next round or timeout:** `next_round()` builds fresh tiles. If the timer runs out mid-drag, the drag is cancelled cleanly.



the drag is cancelled cleanly.

How to test it

1. Run `python main.py` and drag a tile from the top row into the first rack slot. The input box should show that letter.
2. Fill the rack with the correct word and click SUBMIT. You should get CORRECT, +1 score and a new word with fresh tiles.
3. Drag a rack tile onto another rack tile. They should swap, and the input text should reorder.
4. Drag a rack tile back up to the top row. The input box should lose that letter.
5. Drop a tile in empty space, for example near the timer. It should snap back to where it started.
6. Build a wrong word and submit. You should see WRONG GUESS, a cleared box, tiles back in the top row and the same word.
7. Click SUBMIT with an empty rack. You should see "Type a word before submitting!" with no penalty.
8. Click HINT while tiles are in the rack. The hint letters and the -1 point should still work.
9. Start a drag and hold it until the timer hits 0. You should get TIME'S UP, a new word and no stuck tile.

One behavior to know about

Clicking a tile also "clicks away" from the text box, which is the original `TextBox` behavior. After using tiles, click the input box before typing. Enter still submits either way.

For your video, drag the tiles into the rack, submit, then show the hint and the timer. 📺 That covers all four tasks.

 **Game engine**
PY

Download

     2 hours ago

